

1 Abstract

This report details the systematic analysis of high-resolution High-Performance Liquid Chromatography (HPLC) fraction-level datasets comprising 10.8 million rows across 32 runs. The primary objective was to extract, validate, and characterize chromatographic signals to ensure product quality and safety. Due to critical gaps in concentration metadata ('Conc_B_ml') and sample codes, the analysis relied exclusively on UV absorbance signals ('UV_1_280_mAU', 'UV_2_260_mAU') and volume proxies to identify analyte peaks. The workflow involved rigorous data cleaning to remove statistical outliers and negative artifacts, followed by automated peak detection using derivative thresholds and Signal-to-Noise Ratio (SNR) criteria. Subsequent integration calculated peak areas and retention times, while run-level aggregation generated a comprehensive inventory. This report summarizes the methodology, data quality outcomes, and the final inventory of detected peaks, establishing a foundation for downstream purity assessment despite the absence of direct concentration data.

2 Introduction

Chromatographic analysis is fundamental to assessing the purity and identity of complex mixtures. In this study, we analyzed a large-scale dataset containing 10.8 million individual fraction records derived from 32 HPLC runs. The dataset provided high-resolution UV absorbance data at two wavelengths (280 nm and 260 nm) but suffered from significant metadata gaps, specifically the absence of concentration values ('Conc_B_ml') and sample identification codes. Furthermore, the raw data contained numerous artifacts, including negative volume and absorbance values indicative of sensor noise or processing errors. The motivation for this analysis was to rigorously filter these artifacts and detect all significant chromatographic peaks to generate a reliable inventory of analyte retention times, areas, and shapes. Without robust peak detection and artifact removal, downstream quality control metrics such as purity assessment would be compromised. Therefore, this study focuses on developing a robust pipeline that leverages the high granularity of the fraction-level data to characterize peaks based solely on UV signal intensity and volume mapping, ensuring that only statistically significant features are reported.

3 Methodology

The analysis was conducted in three sequential phases: data cleaning, peak detection and integration, and run-level aggregation. First, a data cleaning phase was performed on the raw 'chromatography_data.csv' file. Robust statistical outlier detection was applied to the 'volume_ml' and UV absorbance columns. Rows were identified for removal if they deviated by more than 3 standard deviations from the mean or contained negative values. This step preserved valid low-volume data while eliminating noise artifacts, resulting in a cleaned dataset sorted by run number and fraction number. Second, peak detection and integration were executed on the cleaned data. Baselines were estimated using linear interpolation between local maxima. Peaks were identified by combining a derivative threshold (slope change ≥ 0.5 mAU/unit volume) with a minimum Signal-to-Noise Ratio (SNR) of 3 (Peak Height / Baseline StdDev). Peak areas were calculated by integrating the signal above the estimated baseline, and a minimum height threshold was applied to filter insignificant peaks. Visualizations were generated to illustrate the baseline estimation, peak distribution, and correlation between UV channels. Finally, run-level aggregation was performed to summarize peak statistics. Data was grouped by 'run_no' to calculate summary metrics including peak count, total area, average peak height, and retention time ranges. Runs with zero peaks or total areas below 10% of the median run area were flagged as anomalies. Concentration quantification was explicitly noted as unavailable due to the missing 'Conc_B_ml' variable.

4 Results and Discussion

The data cleaning phase successfully filtered the 10.8 million raw rows, removing statistically significant outliers and negative artifacts to produce a robust dataset for analysis. The removal of invalid rows ensured that subsequent peak detection algorithms operated on clean signal data, minimizing false positives

caused by noise. In the peak detection phase, the combination of derivative thresholding and SNR filtering effectively isolated significant analyte peaks from the baseline noise. The visualized results in Figure 1 demonstrate the efficacy of the baseline estimation method, where linear interpolation between local maxima provided a stable reference for area integration. The histogram of filtered peak areas (Panel 2) indicates a clear separation between background noise and significant peaks, validating the chosen SNR threshold. Furthermore, the scatter plot of UV_1_280_mAU versus UV_2_260_mAU peak heights (Panel 3) reveals a strong positive correlation, suggesting that the analytes of interest absorb significantly at both wavelengths, which is consistent with protein or nucleic acid content. The run-level aggregation identified specific runs as anomalies due to low signal intensity or lack of peaks, highlighting potential issues with specific batches or instrument performance. While concentration quantification could not be performed due to missing meta-data, the inventory of peak retention times and areas provides a reliable proxy for purity assessment and batch consistency monitoring. The high granularity of the fraction-level data allowed for precise estimation of peak shapes and widths, offering insights into peak resolution that would be lost in coarser time-based sampling.

5 Conclusion

This study successfully demonstrated a robust methodology for extracting and characterizing chromatographic signals from large-scale, high-resolution HPLC datasets despite significant metadata gaps. By rigorously filtering artifacts and applying statistically sound peak detection criteria, we generated a comprehensive inventory of peaks across 32 runs. The analysis confirmed that the UV absorbance signals at 280 nm and 260 nm are sufficient for peak identification and area calculation, even in the absence of direct concentration data. The identification of anomaly runs provides critical feedback for quality control, indicating potential process deviations in specific batches. Future work should focus on acquiring the missing ‘Conc_B_ml’ data to enable absolute quantification and to correlate peak areas with actual concentrations. Until then, the current inventory serves as a vital baseline for monitoring chromatographic consistency and ensuring product quality based on signal integrity and retention profile stability.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_data.csv')

    # Select relevant columns
    columns_to_use = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
        Fraction_number', 'run_no']
    df_subset = df[columns_to_use]

    # Pre-calculate mean and std per run for all target columns
    # This avoids recalculating inside the loop and enables vectorized masking
    target_cols = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']
    stats = {}
    for col in target_cols:
        stats[col] = df_subset.groupby('run_no')[col].agg(['mean', 'std']).to_dict(
            'index')
```

```

# Initialize boolean mask for the entire dataframe based on original data
# We start with a mask of True (keep all) and update it iteratively
mask = pd.Series(True, index=df_subset.index)

rows_removed = 0

for col in target_cols:
    run_stats = stats[col]
    run_data = df_subset.copy() # Use original df_subset for consistent stats
                                # lookup

    for run_no in run_stats.keys():
        mean_val = run_stats[run_no]['mean']
        std_val = run_stats[run_no]['std']

        if pd.isna(std_val) or std_val == 0:
            continue

        # Pre-calculate bounds
        lower_bound = mean_val - 3 * std_val
        upper_bound = mean_val + 3 * std_val

        # Create mask for this run's outliers (negative OR outside bounds)
        # Using original run_data ensures we don't filter out rows already
        # removed by previous columns
        # but we apply the logic to the current subset to count accurately
        # against original
        # Actually, to count unique rows removed correctly against the
        # original set,
        # we should identify outliers in the original df_subset for this run.
        run_subset = df_subset[df_subset['run_no'] == run_no]

        # Identify rows to remove for this column
        is_outlier = (run_subset[col] < 0) | (run_subset[col] < lower_bound) |
                     (run_subset[col] > upper_bound)

        # Update global mask: keep rows that are NOT outliers for this column
        # We only apply the mask if the row is currently being considered (
        # mask is True)
        # But to avoid double counting or complex logic, we can just update
        # the global mask directly.
        # If a row is an outlier for ANY column, it should be removed.
        # So we mark it as False (remove) if it matches any outlier condition.

        # Map run indices to global indices
        run_indices = run_subset.index

        # Create a temporary mask for this column's removal
        col_mask = pd.Series(False, index=df_subset.index)
        col_mask[run_indices] = is_outlier

        # Update global mask: remove if outlier in this column
        mask = mask & ~col_mask

        # Count rows removed in this step (unique rows that became False)
        # Since we are updating the mask cumulatively, we count how many
        # became False now
        removed_in_step = col_mask.sum()
        if removed_in_step > 0:

```

```

        rows_removed += removed_in_step

# Apply the final mask to the original subset
filtered_df = df_subset[mask].copy()

# Sort remaining data
filtered_df = filtered_df.sort_values(by=['run_no', 'Fraction_number']).
    reset_index(drop=True)

# Prepare Report
total_rows_processed = len(df_subset)
final_row_count = len(filtered_df)

report_lines = [
    "Data_Cleaning & Artifact_Filtering_Report",
    "=" * 40,
    f"Total_rows_processed: {total_rows_processed}",
    f"Rows_removed_due_to_statistical_deviation/negativity: {rows_removed}",
    f"Final_row_count: {final_row_count}",
    "",
    "Explanation_of_Statistical_Filtering_Logic:",
    "-_Calculated_mean_and_standard_deviation_for_each_variable_grouped_by_
        run_no.",
    "-_Removed_rows_where_values_were_negative.",
    "-_Removed_rows_where_values_deviated_more_than_3_standard_deviations_from
        the_run-specific_mean.",
    "-_This_approach_preserves_valid_low-volume_data_while_removing_extreme_
        outliers."
]

# Create Markdown table
report_lines.append("")
report_lines.append("|_Metric_|_Value_|")
report_lines.append("|_-----|_-----|")
report_lines.append(f"|_Total_rows_processed_|_{total_rows_processed}_|")
report_lines.append(f"|_Rows_removed_|_{rows_removed}_|")
report_lines.append(f"|_Final_row_count_|_{final_row_count}_|")

report_text = "\n".join(report_lines)

# Save report
with open('/workspace/cleaned_data_report.txt', 'w') as f:
    f.write(report_text)

# Save cleaned dataframe
filtered_df.to_csv('/workspace/cleaned_data.csv', index=False)
###

```

Listing 1: Code_1